

Group 1

Reflect Studio – AI Powered Smart Mirror for Elderly Care

Reflect Studio is a smart magic mirror system designed mainly for elderly users. It provides an AI companion that can talk with elders, show daily information such as time, date, calendar, temperature, reminders, and notifications. It uses Raspberry Pi 4, presence detection, LED lighting, voice interaction, Gemini API, AWS Rekognition for face and emotion recognition, and a Flutter mobile app for loved ones to send reminders and receive alerts.



Team Members

E/21/287 - Perera G.S.H

E/21/229 - Kurera P.A.T

E/21/055 - Bandara K.N.K.L.N.

E/21/253 - Manabandu J.P.G.T.R.

Key Users

User Type	Description
Elderly User	Main person using the mirror
Loved One / Caregiver	Uses mobile app to send reminders and receive alerts

Functionalities

#	Functionality	Description	User Type
1	Presence Detection	Mirror turns on when a person is detected for 5 seconds	Elderly User
2	LED Strip Activation	LED strip lights up when presence is detected	Elderly User
3	Mirror UI Display	Shows time, date, calendar, and temperature	Elderly User
4	Wake Word Detection	“Hello Mirror” activates Mirror Man	Elderly User
5	AI Companion Chat	User talks with AI companion using voice	Elderly User
6	Speech-to-Text	Converts user voice into text	System
7	Gemini API Response	Sends text to Gemini API and gets AI response	System
8	Text-to-Speech	Converts AI response into speech through speaker	System
9	Goodbye Command	“Good bye” hides Mirror Man	Elderly User
10	Face Recognition	Identifies known users using camera and AWS Rekognition	System
11	Emotion Recognition	Detects user emotion using AWS Rekognition	System
12	Negative Emotion Alert	Sends alert to loved ones if negative emotion is detected	Caregiver
13	Mobile App Signup	User signs up using email verification	Caregiver
14	Send Reminder	Caregiver sends scheduled reminders to mirror	Caregiver
15	Send Notification	Caregiver sends one-time notifications to mirror	Caregiver

16	Voice Music Player	Plays, pauses, resumes, and stops songs by voice	Elderly User
----	--------------------	--	--------------

Functional Requirements

#	Functional Requirement	Related Functionality	User Type	Priority
1	The system shall turn on the mirror UI when presence is detected continuously for 5 seconds.	Presence Detection	Elderly User	High
2	The system shall light up the LED strip when presence is detected.	LED Activation	Elderly User	Medium
3	The system shall display time, date, calendar, and temperature on the mirror UI.	Mirror UI	Elderly User	High
4	The system shall activate Mirror Man when the user says "Hello Mirror".	Wake Word	Elderly User	High
5	The system shall hide Mirror Man when the user says "Good bye".	Goodbye Command	Elderly User	Medium
6	The system shall convert user speech into text.	Speech-to-Text	System	High
7	The system shall send converted text to Gemini API and receive a response.	AI Companion	System	High
8	The system shall convert Gemini response text into speech.	Text-to-Speech	System	High
9	The system shall capture user images using the Raspberry Pi camera.	Camera Capture	System	High
10	The system shall perform face recognition using AWS Rekognition.	Face Recognition	System	High
11	The system shall detect emotions using AWS Rekognition.	Emotion Recognition	System	High

12	The system shall notify loved ones through the mobile app when negative emotion is detected.	Alert System	Caregiver	High
13	The mobile app shall allow users to sign up using email verification.	Mobile Signup	Caregiver	High
14	The mobile app shall allow caregivers to send reminders to the mirror.	Reminder System	Caregiver	High
15	The system shall display reminders until their scheduled date and time.	Reminder Display	Elderly User	High
16	The system shall display notifications for 15 seconds when someone is present.	Notification System	Elderly User	Medium
17	The system shall play, pause, resume, and stop music using voice commands.	Music Player	Elderly User	Medium

Non Functional Requirements

No	Non-Functional Requirement	Category	Priority	Measurement Metric
1	The system shall respond to voice commands within 5 seconds under normal conditions	Performance	High	Average response time (seconds)
2	The system shall use HTTPS encryption for all cloud communications	Security	High	HTTPS enabled
3	User face data and alerts shall be securely stored in AWS cloud services	Security	High	Secure cloud storage compliance
4	The system shall maintain stable operation for long durations without crashing	Reliability	Medium	System uptime percentage
5	The smart mirror UI shall be simple and readable for elderly users	Usability	High	User feedback and usability testing
6	The system shall detect emotions with at least 80% accuracy	Accuracy	Medium	Emotion detection accuracy percentage
7	The system shall automatically activate within 5 seconds after detecting a user presence	Performance	Medium	Activation response time
8	The system shall support multilingual voice interaction	Compatibility	Medium	Number of supported languages
9	The system shall continue operating under normal home Wi-Fi conditions	Reliability	Medium	Successful operation rate
10	The system shall support future feature expansion through modular architecture	Scalability	Low	Ease of integration of new modules

Domain Requirements

Elderly loneliness and social isolation have become major global health concerns. As people grow older, many experience reduced social interaction due to retirement, loss of loved ones, physical limitations, or living alone. According to the World Health Organization, social isolation and loneliness have serious impacts on physical health, mental wellbeing, quality of life, and longevity.

Health experts and researchers have identified loneliness as a significant risk factor for depression, anxiety, heart disease, stroke, cognitive decline, and dementia among older adults. Studies show that social isolation can increase the risk of dementia by nearly 50–60%.

The Centers for Disease Control and Prevention also states that loneliness in older adults is associated with higher risks of depression, anxiety, diabetes, dementia, and early death.

Domain specialists in elderly care emphasize the importance of maintaining social connection, emotional support, and regular interaction for healthy aging. The WHO further highlights that improving social connection can reduce health risks and improve overall wellbeing in older adults.

Reflect Studio addresses this domain problem by providing an AI-powered smart mirror that offers companionship, reminders, emotional monitoring, and caregiver notifications to help reduce loneliness and support elderly wellbeing.

#	Domain Requirement	Domain	Reason / Justification
1	The system should support elderly-friendly interaction methods	Elderly Care	Elderly users may struggle with complex interfaces
2	The system should provide voice-based communication	Smart Home	Easier interaction for elderly users
3	The system should detect negative emotional states	Healthcare Assistance	Helps caregivers identify emotional distress
4	The system should notify caregivers about important emotional events	Elderly Care	Enables quick caregiver response
5	The system should support multilingual communication	Social Accessibility	Elderly users may prefer native languages

6	The system should operate continuously inside a home environment	Smart Home	The mirror is intended for daily household use
7	The system should maintain user privacy when handling face and emotion data	Healthcare Assistance	Protects sensitive personal information
8	The system should support hands-free interaction	AI Assistant	Elderly users may have mobility limitations

Feasibility Analysis

Technical Feasibility

The proposed AI Smart Magic Mirror system is technically feasible because the required hardware, software, APIs, and development technologies are available and compatible with each other.

The system is mainly developed using Python programming language together with hardware and cloud technologies. The main processing unit of the system is the Raspberry Pi 4, which provides sufficient processing power for running the mirror interface, AI assistant, sensors, and communication modules.

Technologies and Components Used

- Python programming language
- Raspberry Pi 4 as the main controller
- Raspberry Pi Camera Module for image capturing
- Raspberry Pi microphone for voice input
- Speaker system for audio output
- LED strip for smart lighting effects
- Gemini API for AI conversation handling
- AWS Rekognition for face recognition and emotion detection
- Flutter framework for mobile application development

System Features

- Presence detection system
- Smart mirror user interface
- AI companion ("Mirror Man")
- Voice-controlled music player
- Reminder and notification system
- Emotion recognition and alert system
- Mobile application integration

The integration between hardware and software components is achievable because the APIs and libraries required for the project are publicly available and well documented. The system architecture supports real-time processing, voice interaction, and cloud communication.

Therefore, the project is technically feasible.

Economic Feasibility

The project is economically feasible because the total development cost is reasonable compared to the benefits provided to elderly users and caregivers.

Estimated Costs

- Raspberry Pi 4
- Raspberry Pi Camera Module
- Microphone and speaker setup
- LED strips and sensors
- Mirror display hardware
- Internet and cloud service costs
- Mobile application development costs
- API usage costs for Gemini and AWS Rekognition

Expected Benefits

- Improved elderly companionship and mental well-being
- Reduced loneliness among elderly users
- Real-time monitoring of emotional conditions
- Better communication between elders and family members
- Automated reminder system for daily activities and medication
- Hands-free smart home interaction

The project reduces the need for continuous physical supervision while improving the quality of life for elderly individuals. Since the long-term social and practical benefits are greater than the implementation costs, the project is economically feasible.

Operational Feasibility

The system is operationally feasible because it is designed to be simple, interactive, and user-friendly for elderly users.

The smart mirror automatically activates when a person is detected for 5 seconds. This reduces unnecessary manual interaction and improves usability for elders. The LED strip lighting and voice wake-word system provide a more natural interaction experience.

Operational Features

- Automatic mirror UI activation using presence detection
- Wake-word activation using "Hello Mirror"
- AI assistant appearance and disappearance control
- Voice-based communication system
- Voice-controlled music playback

- Reminder and notification display
- Emotion monitoring and emergency notification system

The mirror interface displays:

- Time
- Date
- Calendar
- Temperature
- Notifications
- Reminders

The mobile application allows family members to:

- Create user accounts
- Verify email accounts using verification codes
- Send reminders and notifications remotely
- Receive emotion-based alerts

Because the system minimizes complex interactions and supports voice-based control, it is highly suitable for elderly users. Therefore, the project is operationally feasible.

Schedule Feasibility

The project is schedule feasible because the development tasks can be completed within the planned project timeline.

The project can be divided into several phases:

1. Requirement gathering and research
2. Hardware setup and testing
3. Mirror UI development
4. AI assistant integration
5. Voice recognition and speech synthesis development
6. Face and emotion recognition implementation
7. Mobile application development
8. Notification and reminder system integration
9. Testing and debugging
10. Final deployment and documentation

The technologies used in the project already provide existing libraries and APIs, which reduces development time. Proper task allocation among team members further supports timely completion.

Therefore, the project is schedule feasible.

Legal Feasibility

The project is legally feasible because it follows standard software development and data privacy regulations.

The system uses cloud services such as Gemini API and AWS Rekognition according to their service policies and licensing agreements. User information such as facial data, voice data, email addresses, and notifications will be securely handled.

Legal Considerations

- User data privacy protection
- Secure storage of user information
- Authorized API usage
- Compliance with software licensing agreements
- Ethical handling of facial recognition data
- Secure authentication and email verification

The project does not use illegal software, unauthorized data collection methods, or restricted technologies. Therefore, the project is legally feasible.

Cultural Feasibility

The project is culturally feasible because it addresses social issues related to elderly care and emotional well-being.

Many elderly individuals experience loneliness and require emotional support. The AI companion system helps elders communicate naturally through voice interaction, making technology more approachable and comfortable.

Cultural Advantages

- Encourages emotional support for elders
- Improves communication with family members
- Reduces isolation and loneliness
- Easy-to-use voice interaction system
- User-friendly and familiar interface

The system supports users with limited technical knowledge and promotes acceptance of smart technologies in elderly care environments.

Therefore, the project is culturally feasible.

Deliverables

The following deliverables will be provided during the project:

1. AI Smart Magic Mirror System
2. Mirror User Interface
3. AI Companion ("Mirror Man")
4. Presence Detection System
5. Voice Recognition and Speech System
6. Emotion Recognition Module
7. Face Recognition Module
8. Reminder and Notification System
9. Voice-Controlled Music Player
10. Mobile Application Developed Using Flutter
11. User Authentication and Email Verification System
12. Cloud API Integrations
13. Hardware Integration Setup
14. Source Code Documentation
15. Testing and Evaluation Reports
16. User Manual
17. Final Project Report
18. Project Presentation

Full Feasibility Analysis

The feasibility analysis indicates that the AI Smart Magic Mirror project for elderly care is practical, achievable, and beneficial.

- Technically, the system can be developed using available hardware, software, APIs, and programming technologies.
- Economically, the long-term benefits for elderly care and emotional support justify the development costs.
- Operationally, the system is simple and convenient for elderly users through automation and voice interaction.
- Schedule-wise, the project can be completed within the planned timeline using existing tools and libraries.
- Legally, the project complies with data privacy, API usage, and software licensing requirements.
- Culturally, the system supports elderly well-being and encourages social and emotional support.

Overall, the project is feasible and suitable for implementation as an intelligent elderly care assistance system.

In-Scope and Out-of-Scope

In-Scope

The proposed AI Smart Magic Mirror system focuses on providing intelligent assistance, emotional companionship, and smart interaction features for elderly users. The following functionalities and components are included within the scope of this project.

Hardware Architecture

The system uses Raspberry Pi 4 as the main processing unit to manage all smart mirror operations and hardware integrations.

Included Hardware Components

- Raspberry Pi 4 for central processing
- Raspberry Pi Camera Module for image capturing
- Raspberry Pi microphone for voice input
- Speaker system for audio output
- LED strip for smart environment lighting
- Presence detection sensor for automatic mirror activation

The mirror interface activates automatically when a person is detected for more than 5 seconds. At the same time, the LED strip lights up to improve user interaction and visibility.

AI and Interaction System (Mirror Man)

The system includes an AI-powered virtual companion called “Mirror Man,” specially designed to interact naturally with elderly users.

Voice Interface

- Wake-word detection using the phrase “Hello Mirror”
- “Goodbye” voice command to deactivate the AI avatar
- Real-time voice interaction support

Conversational AI

The system integrates the Gemini API to provide intelligent conversations and companionship for elderly users.

The AI workflow includes:

1. Voice input captured through the microphone
2. Speech-to-Text (STT) conversion
3. Processing user requests using Gemini API
4. Text-to-Speech (TTS) generation
5. Audio response through the speaker system

This feature enables natural communication between the elderly user and the smart mirror.

Computer Vision and Emotion Recognition

The system uses AWS Rekognition for computer vision functionalities.

Included Features

- Face recognition and identification
- Emotion detection and analysis
- Detection of negative emotional states such as sadness or distress

When a negative emotion is detected, the system sends alerts and notifications to registered loved ones through the mobile application.

Elder-Centric User Interface (UI/UX)

The mirror interface is designed to be simple, clear, and user-friendly for elderly users.

Displayed Information

- Time
- Date
- Calendar events
- Local temperature
- Notifications
- Reminders

Dynamic Notification System

- Notifications remain displayed until a user appears in front of the mirror
- Once detected, notifications automatically disappear after 15 seconds

Reminder System

Family members can send reminders through the mobile application. These reminders remain visible on the mirror until the scheduled date and time.

Voice-Controlled Music System

The project includes a Python-based voice-assisted music player.

Supported Commands

- "Play [song name]"
- Pause music
- Resume music
- Stop music

This feature allows elderly users to control music playback using simple voice commands without physical interaction.

Cross-Platform Integration

The system includes a mobile application developed using Flutter for family members and caregivers.

Mobile Application Features

- User registration and login
- Email verification using OTP codes
- Sending reminders to the mirror
- Receiving emotional alerts and notifications
- Managing user profiles and settings

The application provides remote communication and monitoring support between elders and their loved ones.

Cloud and Backend Services

The project includes cloud-based services and backend functionalities for secure communication and data management.

Included Backend Features

- Secure user authentication
 - Storage of reminders and notifications
 - User profile management
 - Cloud communication between the mirror and mobile app
 - Integration with Gemini API and AWS Rekognition services
-

Out-of-Scope

The following features and functionalities are not included in the current version of the project.

Medical Diagnostic Features

Although the system can detect emotions and identify negative emotional states, it does not provide:

- Medical diagnoses
- Mental health assessments
- Psychological evaluations
- Clinical recommendations

The system is designed only for emotional monitoring and companionship support.

Emergency Service Integration

The system can notify family members or loved ones during emotional distress situations. However, it does not:

- Automatically contact emergency services
 - Dial emergency numbers such as 119 or 911
 - Provide direct emergency response handling
-

Physical Health Monitoring

The current version of the project does not include wearable or medical sensor integration.

Excluded Features

- Heart rate monitoring
 - Blood pressure monitoring
 - Oxygen level monitoring
 - Body temperature monitoring
 - Smart wearable device integration
-

Video Calling Functionality

Direct video communication through the mirror interface is not included in this release.

Excluded Communication Features

- Peer-to-peer video calls
- Real-time video conferencing
- Integrated video chat system

These features may be considered for future versions of the project.

Summary

The AI Smart Magic Mirror project mainly focuses on providing intelligent companionship, emotional monitoring, voice interaction, and remote family connectivity for elderly users. The system combines AI technologies, computer vision, cloud services, and IoT hardware to create a user-friendly smart assistance platform while keeping the project scope manageable and achievable within the current development phase.

Week 2- Use Case Descriptions and Use Case Diagrams

Prioritised Functionalities

1. Detect Presence and Turn On Mirror UI
2. Activate Mirror Man using Wake Word
3. AI Conversation with Mirror Man
4. Voice Assisted Music Player
5. Face Recognition
6. Emotion Recognition and Alert Loved Ones
7. Mobile App Sign Up and Email Verification
8. Send Reminder / Notification to Mirror

Main Actors: Elderly User, Care Giver/ Loved ones

Elderly User

The Elderly User is the primary user of the Reflect Studio Magic Mirror System. They interact with the smart mirror to view daily information such as time, date, calendar, and temperature, communicate with the AI companion (Mirror Man), receive reminders and notifications, and control music using voice commands. The system is designed to support their emotional wellbeing, reduce loneliness, and assist with daily activities.

Caregiver / Loved One

The Caregiver / Loved One is a secondary user who supports the elderly user remotely through the mobile application. They can create accounts, verify email addresses, send reminders and notifications to the mirror, and receive alerts when negative emotions are detected. Their role is to help monitor and support the wellbeing of the elderly user.

Use Case 01 — Detect Presence and Turn On Mirror UI

Field	Description
Use Case ID	UC01
Use Case Name	Detect Presence and Turn On Mirror UI
Primary Actor	Elderly User
Supporting Components	Presence Sensor, Raspberry Pi, LED Strip
Preconditions	System is powered on and running
Trigger	A person stands in front of the mirror
Main Flow	1. Presence sensor detects a person. 2. System checks whether the person remains for 5 seconds continuously. 3. Raspberry Pi confirms valid presence. 4. LED strip turns on. 5. Mirror UI is displayed. 6. Time, date, calendar, and temperature are shown on the mirror.
Postconditions	Mirror UI becomes active and ready for interaction
Alternate Flow	If the person leaves before 5 seconds, the mirror UI remains off.
Exceptions	If the sensor fails, the system cannot detect presence and the UI will not activate.
Created By	E/21/287

Use Case02 — Activate/Deactivate Mirror Man using Voice Commands

Field	Description
Use Case ID	UC02
Use Case Name	Activate/Deactivate Mirror Man using Voice Commands
Primary Actor	Elderly User

Supporting Components	Microphone, Speech Recognition Module, Raspberry Pi
Preconditions	Mirror UI is active and microphone is functioning
Trigger	User says "Hello Mirror" or "Good bye"
Main Flow	1. System continuously listens for voice commands. 2. User says "Hello Mirror". 3. Speech recognition module detects the wake word. 4. Raspberry Pi validates the command. 5. Mirror Man avatar appears on the screen. 6. AI conversation mode becomes active. 7. User interacts with Mirror Man. 8. User says "Good bye". 9. Speech recognition module detects the goodbye command. 10. Raspberry Pi validates the command. 11. Mirror Man avatar disappears from the screen. 12. AI conversation mode is deactivated.
Postconditions	Mirror Man is activated or deactivated according to the user's voice command
Alternate Flow	If an unrelated word is detected, Mirror Man remains in its current state.
Exceptions	If microphone input is unclear, unavailable, or speech recognition fails, the command will not be executed.
Created By	E/21/287

Use Case 03: Face Recognition

Field	Details
Use Case Name	Face Recognition
Primary Actor	Elderly User

Description	This use case describes how the smart mirror identifies elderly users using the Raspberry Pi Camera Module and AWS Rekognition services.
Pre-conditions	1. The Smart Mirror system is powered on. 2. Raspberry Pi Camera Module is connected and functioning. 3. AWS Rekognition services are available. 4. User face data is registered in the system database.
Main Flow	1. A person stands in front of the smart mirror. 2. Presence detection activates the mirror interface. 3. The Raspberry Pi Camera Module captures an image. 4. The image is processed using Python image-processing modules. 5. The image is sent to AWS Rekognition. 6. AWS Rekognition detects and compares the face with registered user data. 7. The system identifies the user. 8. Personalized mirror interactions and services are activated.
Post-conditions	1. User is successfully identified. 2. Personalized features are enabled. 3. User interaction history may be updated in the system.
Exceptions	E1: Camera module failure. E2: AWS Rekognition service unavailable. E3: Face not recognized in the database.

	E4: Poor lighting conditions reduce recognition accuracy.
Main Functions	- Capture user images using Raspberry Pi Camera Module. - Detect and identify faces. - Recognize registered users. - Support personalized interactions.
Technologies Used	Raspberry Pi 4, Raspberry Pi Camera Module, AWS Rekognition, Python
Expected Outcome	The mirror automatically recognizes users and provides personalized smart assistance and interactions.

Use Case 04: Emotion Recognition and Alert Loved Ones

Field	Details
Use Case Name	Emotion Recognition and Alert Loved Ones
Primary Actor	Elderly User
Description	This use case describes how the system analyzes user emotions using facial recognition and sends alerts to loved ones when negative emotional conditions are detected.
Pre-conditions	1. Camera module is operational. 2. AWS Rekognition emotion analysis services are available. 3. Mobile application is connected to the backend system. 4. Loved ones are registered in the mobile application.

Main Flow	1. User stands in front of the mirror. 2. Camera captures facial images. 3. Images are processed using Python. 4. AWS Rekognition analyzes facial emotions. 5. The system checks for negative emotional states such as sadness or distress. 6. If a negative emotion is detected, an alert notification is generated. 7. Notification is sent to registered loved ones through the Flutter mobile application. 8. Family members receive emotional alerts on their devices.
Post-conditions	1. Emotional state is analyzed successfully. 2. Loved ones receive alerts during negative emotional conditions. 3. Emotional monitoring logs may be updated in the system.
Exceptions	E1: Emotion detection failure. E2: Internet connection failure. E3: Notification delivery failure. E4: AWS Rekognition service unavailable.
Main Functions	- Detect user emotions through facial analysis. - Identify negative emotional states. - Trigger alert notifications. - Support emotional monitoring for elderly care.
Negative Emotion Alerts	- Notification is generated. - Alert is sent to loved ones. - Family members are informed through the mobile application.
Technologies Used	AWS Rekognition Emotion Detection, Python, Flutter Mobile Application, Notification Services
Expected Outcome	Family members can monitor the emotional well-being of elderly users and provide support when necessary.

Use Case UC05 — AI Conversation with Mirror Man

Field	Details
Use Case ID	UC03
Use Case Name	AI Conversation with Mirror Man
Prepared By	E21229 – Anna Kurera
Actor	Elderly User
Pre-conditions	Mirror UI is active and Mirror Man is activated using wake word
Main Flow	1. User speaks to Mirror Man 2. System captures voice input 3. Speech is converted to text 4. Text is sent to Gemini API 5. AI response is generated 6. Response is converted to speech 7. Mirror Man replies to the user
Post-conditions	User receives an AI-generated voice response
Exceptions / Alternate Flows	Internet connection unavailable → AI response cannot be generated

Use Case UC06 — Voice Assisted Music Player

Field	Details
Use Case ID	UC04
Use Case Name	Voice Assisted Music Player
Prepared By	E21229- Anna Kurera
Actor	Elderly User
Pre-conditions	Presence of someone in front of the mirror should be detected and internet connection is available
Main Flow	1. User gives a voice command to play music 2. System converts speech to text 3. Music request is processed 4. YouTube API searches for music 5. Selected music is played through speaker
Post-conditions	Requested music is played successfully
Exceptions / Alternate Flows	Song not found or internet unavailable

Use Case UC07: Mobile App Sign Up and Email Verification

Field	Details
Use Case ID	UC-07
Use Case Name	Mobile App Sign Up and Email Verification
Primary Actor	Primary User
Description	This use case describes how a user creates an account in the ReflectStudio mobile application and verifies the account using email authentication services.
Pre-conditions	1. User has the ReflectStudio mobile application installed. 2. Device is connected to the internet. 3. AWS cloud services are available and running.

Main Flow	1. User opens the Flutter mobile application. 2. User selects the “Sign Up” option. 3. User enters required details such as email and password. 4. The application sends the registration request to AWS Cognito. 5. AWS Cognito creates a temporary user account. 6. AWS SES sends a verification code or verification email to the user’s email address. 7. User enters the verification code or confirms the email. 8. The system verifies the account information. 9. User profile details are stored securely in the AWS database. 10. User account becomes active and accessible.
Post-conditions	1. User account is successfully created. 2. Email address is verified. 3. User data is securely stored in the cloud database. 4. User can access the mobile application features.
Exceptions	E1: AWS service timeout or network failure. E2: User already exists in AWS Cognito. E3: Invalid or expired verification code. E4: Internet connection unavailable during verification process.
Technologies Used	Flutter, AWS Cognito, AWS SES, AWS Database, Python Backend APIs

Use Case 08: Send Reminder / Notification to Mirror

Field	Details
Use Case ID	UC-08
Use Case Name	Send Reminder / Notification to Mirror
Primary Actor	Primary User
Description	This use case describes how family members or caregivers send reminders and notifications from the mobile application to the AI Smart Mirror system.

Pre-conditions	1. User is successfully authenticated. 2. Smart Mirror is connected to the internet. 3. Mirror device is linked to the user's AWS account. 4. AWS backend services are available.
Main Flow	1. User opens the Flutter mobile application. 2. User navigates to the reminder or notification section. 3. User enters reminder details including message text and scheduled date/time. 4. The application sends the data through AWS API Gateway. 5. AWS Lambda processes the request. 6. Reminder data is stored in the AWS cloud database. 7. The Raspberry Pi Smart Mirror retrieves the updated reminder data through polling or push notification services. 8. The mirror UI displays the reminder or notification at the scheduled time. 9. Notifications remain visible until a person is detected in front of the mirror. 10. Once presence is detected, notifications automatically disappear after 15 seconds.
Post-conditions	1. Reminder is successfully stored in the cloud database. 2. Reminder is synchronized with the Smart Mirror. 3. Notification is displayed correctly on the mirror interface. 4. Reminder remains active until the scheduled date expires.
Exceptions	E1: Failure to connect to AWS cloud services. E2: Raspberry Pi fails to retrieve updated reminder data. E3: Invalid reminder timestamp or message format. E4: Internet connection interrupted during synchronization.
Technologies Used	Flutter, AWS API Gateway, AWS Lambda, AWS Database Services, Python, Raspberry Pi 4

Trello board link:

<https://trello.com/b/J9wYLxn/my-trello-board>

Week 3: Class Diagram

Understanding the system first

Your project is **Reflect Studio – AI Powered Smart Mirror for Elderly Care**. It has an elderly user, caregivers/loved ones, admin/developer, and guest/family members.

Your main functionalities include:

- Presence detection
- Mirror UI display
- Wake word detection
- AI companion chat
- Speech-to-text
- Gemini API response
- Text-to-speech
- Face recognition
- Emotion recognition
- Negative emotion alert
- Mobile app signup
- Send reminders/notifications
- Voice music player

So your class diagram must represent these main parts.

Identifying possible classes

In class diagrams, we mainly look for **nouns** from the use cases.

From your use cases, possible classes are:

User-related classes

- User
- ElderlyUser
- Caregiver
- Admin

- Guest

Mirror system classes

- SmartMirrorSystem
- MirrorUI
- PresenceSensor
- LEDStrip
- RaspberryPi
- CameraModule
- Microphone
- Speaker

AI and voice classes

- MirrorMan
- VoiceCommand
- SpeechToTextService
- TextToSpeechService
- GeminiAPIService

Face/emotion recognition classes

- FaceRecognitionService
- EmotionRecognitionService
- AWSRekognitionService
- EmotionAlert

Reminder/notification classes

- Reminder
- Notification
- ReminderService
- NotificationService

Mobile app/backend classes

- MobileApp
- AuthenticationService
- EmailVerificationService
- CloudDatabase

- `AWSBackendService`

Music classes

- `MusicPlayer`
- `YouTubeAPIService`
- `Song`

These classes directly come from your Week 2 use cases such as presence detection, Mirror Man activation, AI conversation, face recognition, emotion recognition, mobile signup, and reminder/notification sending.

Grouping the classes into domains

In your Google Doc, you can write like this:

1. User Domain

Contains:

User
ElderlyUser
Caregiver
Admin
Guest

Purpose: represents people who interact with the system.

2. Mirror Hardware Domain

Contains:

SmartMirrorSystem
PresenceSensor
LEDStrip
CameraModule
Microphone
Speaker
RaspberryPi

Purpose: represents the physical components of the mirror.

3. AI Interaction Domain

Contains:

MirrorMan
VoiceCommand
SpeechToTextService
GeminiAPIService
TextToSpeechService

Purpose: handles voice interaction and AI conversation.

4. Vision and Emotion Domain

Contains:

FaceRecognitionService
EmotionRecognitionService
AWSRekognitionService
EmotionAlert

Purpose: handles face recognition, emotion detection, and alerts.

5. Reminder and Notification Domain

Contains:

Reminder
Notification
ReminderService
NotificationService

Purpose: handles messages sent from caregivers to the mirror.

6. Mobile App and Cloud Domain

Contains:

MobileApp
AuthenticationService
EmailVerificationService
CloudDatabase
AWSBackendService

Purpose: handles mobile app signup, login, reminders, notifications, and cloud communication.

Defining important classes with attributes and methods

Class 1: User

User

- userId: String
- name: String
- email: String
- role: String

+ login(): boolean
+ logout(): void
+ updateProfile(): void

This can be the parent class.

Class 2: ElderlyUser

ElderlyUser

- age: int
- preferredLanguage: String
- emergencyContact: String

+ viewMirrorUI(): void
+ speakCommand(command: String): void
+ receiveReminder(reminder: Reminder): void

ElderlyUser inherits from User.

Class 3: Caregiver

Caregiver

- caregiverId: String

```
- phoneNumber: String
-----
+ sendReminder(reminder: Reminder): void
+ sendNotification(notification: Notification): void
+ receiveEmotionAlert(alert: EmotionAlert): void
```

Caregiver also inherits from User.

Class 4: SmartMirrorSystem

```
SmartMirrorSystem
-----
- systemId: String
- isActive: boolean
- currentUser: ElderlyUser
-----
+ startSystem(): void
+ turnOnUI(): void
+ turnOffUI(): void
+ processVoiceCommand(command: VoiceCommand): void
```

This is the main class that connects the whole system.

Class 5: PresenceSensor

```
PresenceSensor
-----
- sensorId: String
- detectionTime: int
- isPersonDetected: boolean
-----
+ detectPresence(): boolean
+ validateFiveSecondPresence(): boolean
```

This matches your use case where the mirror turns on only if a person is detected continuously for 5 seconds.

Class 6: MirrorUI

MirrorUI

```
-----  
- time: String  
- date: String  
- temperature: double  
- isVisible: boolean  
-----  
+ displayTime(): void  
+ displayDate(): void  
+ displayTemperature(): void  
+ displayReminder(reminder: Reminder): void  
+ displayNotification(notification: Notification): void
```

This represents the mirror screen that shows time, date, calendar, temperature, reminders, and notifications.

Class 7: MirrorMan

MirrorMan

```
-----  
- avatarName: String  
- isActivated: boolean  
-----  
+ activate(): void  
+ deactivate(): void  
+ speakResponse(response: String): void
```

This matches the “Hello Mirror” and “Good bye” use case.

Class 8: VoiceCommand

VoiceCommand

```
-----  
- commandId: String  
- commandText: String  
- commandType: String  
-----  
+ identifyCommandType(): String  
+ executeCommand(): void
```

Examples:

Hello Mirror
Good bye
Play song
Pause music
Stop music

Class 9: **SpeechToTextService**

SpeechToTextService

- language: String

+ convertSpeechToText(audioInput: String): String

This converts user voice into text before sending it to Gemini.

Class 10: **GeminiAPIService**

GeminiAPIService

- apiKey: String
- modelName: String

+ generateResponse(prompt: String): String

This represents the Gemini API part of the AI conversation. Your use case says user speech is converted to text, sent to Gemini API, and the response is converted back to speech.

Class 11: **TextToSpeechService**

TextToSpeechService

- voiceType: String

+ convertTextToSpeech(text: String): Audio

Class 12: **CameraModule**

CameraModule

- cameraId: String

- resolution: String
- isAvailable: boolean

+ captureImage(): Image

Used for face recognition and emotion recognition.

Class 13: AWSRekognitionService

AWSRekognitionService

-
- accessKey: String
 - region: String

+ recognizeFace(image: Image): String
+ detectEmotion(image: Image): String

Class 14: FaceRecognitionService

FaceRecognitionService

-
- recognitionStatus: boolean

+ identifyUser(image: Image): ElderlyUser
+ compareWithDatabase(image: Image): boolean

Class 15: EmotionRecognitionService

EmotionRecognitionService

-
- detectedEmotion: String
 - confidenceLevel: double

+ analyzeEmotion(image: Image): String
+ isNegativeEmotion(emotion: String): boolean

Class 16: EmotionAlert

EmotionAlert

-
- alertId: String
 - emotionType: String

- timestamp: Date

- message: String

+ createAlert(): void

+ sendToCaregiver(caregiver: Caregiver): void

This matches your use case where negative emotion detection sends alerts to loved ones.

Class 17: Reminder

Reminder

- reminderId: String

- message: String

- scheduledDateTime: Date

- status: String

+ scheduleReminder(): void

+ markAsDisplayed(): void

+ expireReminder(): void

Class 18: Notification

Notification

- notificationId: String

- message: String

- createdTime: Date

- displayDuration: int

+ display(): void

+ hideAfter15Seconds(): void

Your use case says notifications disappear after 15 seconds once presence is detected.

Class 19: MobileApp

MobileApp

- appVersion: String

- platform: String

+ registerUser(): void
+ verifyEmail(): boolean
+ sendReminder(reminder: Reminder): void
+ sendNotification(notification: Notification): void
+ receiveAlert(alert: EmotionAlert): void

Class 20: **MusicPlayer**

MusicPlayer

- currentSong: String
- isPlaying: boolean
- volume: int

+ playSong(songName: String): void
+ pauseMusic(): void
+ resumeMusic(): void
+ stopMusic(): void

This matches the voice-assisted music player use case.

E21229

MirrorMan
-avatar name: String - isActivated: boolean
+ activate(): void + deactivate(): void + speakResponse(response: String): void

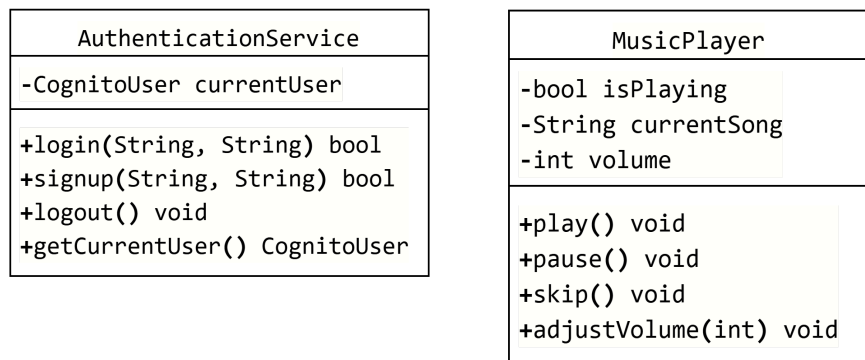
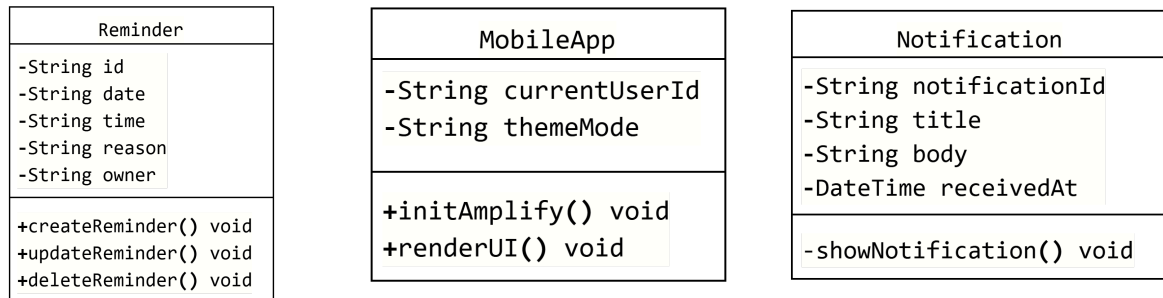
SpeechToTextService
- language: String
+ convertSpeechToText(audioInput: String): String

GeminiAPIService
- apiKey: String - modelName: String
+ generateResponse(prompt: String): String

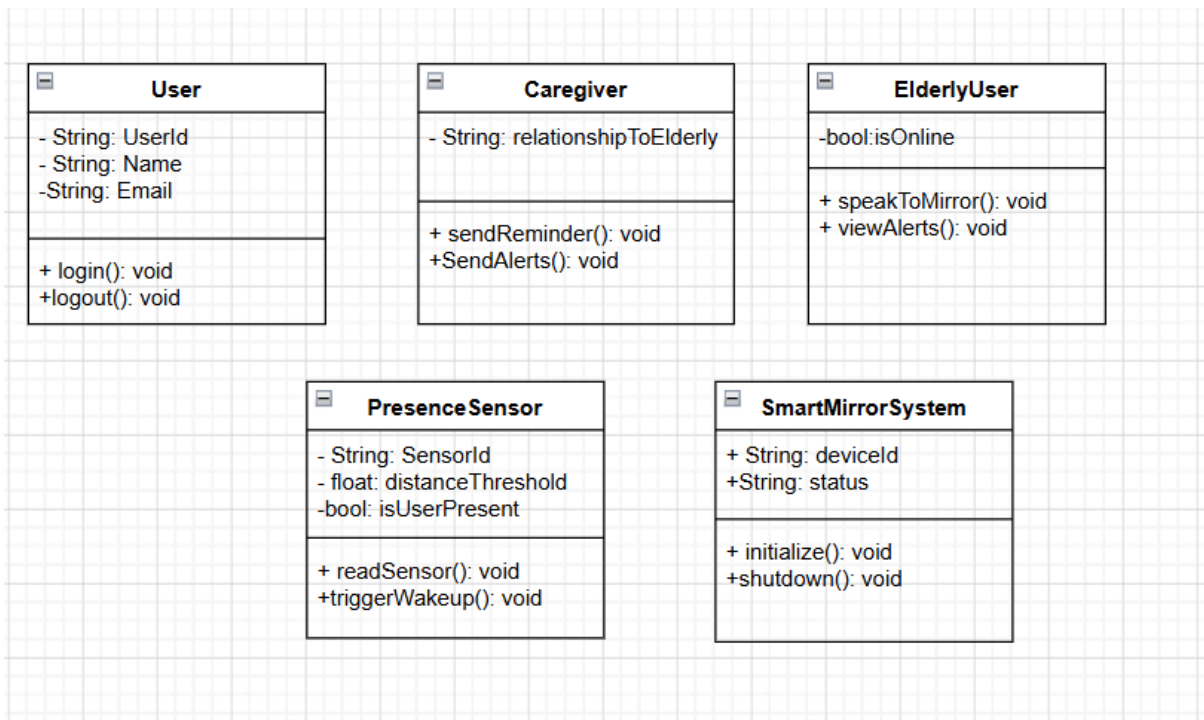
VoiceCommand
- commandId: String - commandText: String - commandType: String
+ identifyCommandType(): String + executeCommand(): void

TextToSpeechService
- voiceType: String
+ convertTextToSpeech(text: String): Audio

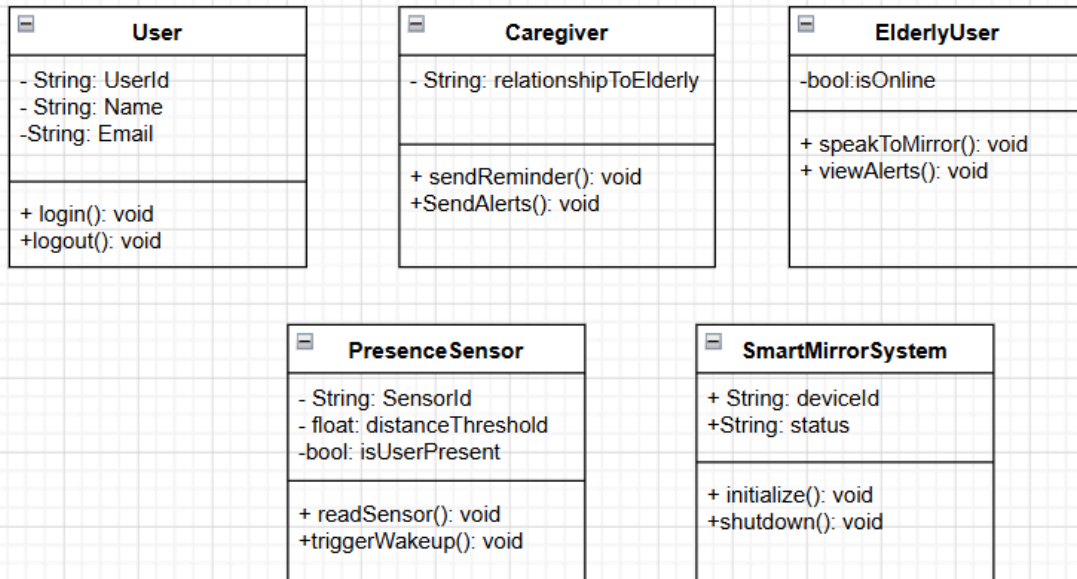
E/21/055



E/21/287



E/21/253



Step 3: Draw the class diagram

